TECHWEEK · COLLÈGE LASALLE · AUTOMNE 2026

Conducteur d'atelier
Construire un agent IA pédagogique local

10 étapes, 3 extensions, 2 heures, aucune clé API

De la première réponse du modèle au TP qui se débloque tout seul

10	6	0
étapes numérotées, une commande par étape	démonstrations de contrôle prêtes à montrer	compte, clé API ou port ouvert

CE DOCUMENT NE CONTIENT AUCUN CODE PYTHON

Vous avez déjà la solution complète dans les fichiers etape1.py à etape10_ressources.py, app_final.py, app_tp.py et app_maths.py. Ce conducteur donne autre chose : le sens de chaque étape, la commande exacte, le résultat attendu, le lien avec la suite, et la réponse à l'objection « on devrait pouvoir poser n'importe quelle question ». L'étape 8, qui manquait, est fournie à part dans etape8.py.

Version du 1er novembre 2026 · à utiliser pendant la séance, pas à lire avant

Mode d'emploi

Trois lectures possibles du même document

Ce conducteur est un outil de pilotage. Il ne remplace pas les fichiers de l'atelier : il dit quoi faire, dans quel ordre, ce qu'on doit voir, et pourquoi c'est là.

Moment	Ce que vous lisez	Ce que vous faites
J-7	Sections « Le pitch et l'exploit », « La carte du parcours », « Avant de commencer », annexe D	Installer, tester le hors-ligne, préparer la clé USB.
Pendant	Section « Les dix étapes » et « Le conducteur 2 h »	Vous suivez la colonne « commande », vous montrez la colonne « attendu ».
Après	Sections « On devrait pouvoir poser n'importe quelle question », « Les démonstrations de contrôle », « Dépannage express »	Vous répondez aux objections, vous rejouez les démonstrations, vous dépannez.

Les cinq rubriques, toujours dans le même ordre

- **COMMANDE** — ce que vous tapez, tel quel, dans le dossier mon_agent_react.
- **RÉSULTAT ATTENDU** — ce que la console ou le navigateur doit afficher. Si ce n'est pas ça, allez à la section « Dépannage ».
- **POURQUOI** — l'utilité réelle de l'étape pour l'agent. C'est ce que vous dites à voix haute.
- **LIEN** — ce que l'étape suivante ajoute, et ce qui casserait sans celle-ci.
- **PIÈGE / CONTRÔLE** — l'erreur classique, et la question à poser aux étudiants pour vérifier qu'ils ont compris.

> **LA SEULE PHRASE À RETENIR**
>
> Prompt = politesse, Code = police. Le modèle propose des mots ; Python décide de ce qui est vrai, note, daté, corrigé ou débloqué. Tout le reste du document découle de cette phrase.

Ce dont vous avez besoin

Élément	Version / taille	Rôle	Où
Python	3.10 ou plus	Exécute l'agent, le serveur, les outils	python.org (cocher Add to PATH)
Ollama	application de bureau	Fait tourner le modèle en local	ollama.com/download
llama3.2	2 Go, version latest	Le modèle de langage de tout l'atelier	ollama pull llama3.2:latest
mon_agent_react	le dossier de l'atelier	Contient les 13 scripts, cours/, Tps/, static/	votre clé USB ou le dépôt
Bibliothèques	requests, pypdf, transformers	Réseau local, lecture PDF, tokens	installer.bat

Trois conventions de ce document

- Toutes les adresses sont en 127.0.0.1, jamais localhost : c'est la même machine, sans détour DNS.
- Les commandes se tapent dans le dossier mon_agent_react. Si vous êtes ailleurs, faites d'abord cd mon_agent_react.
- Les blocs « résultat attendu » sont des extraits réels, pas des illustrations. Les accents peuvent s'afficher bizarrement dans PowerShell : les fichiers, eux, sont corrects.

Sommaire

Le sommaire se met à jour tout seul à l'ouverture : si les numéros de page sont absents, faites Ctrl+A puis F9 (ou clic droit → Mettre à jour les champs).

Mode d'emploi 2
Les cinq rubriques, toujours dans le même ordre 2
Ce dont vous avez besoin 2
Trois conventions de ce document 2
Le pitch et l'exploit 5
En soixante secondes 5
L'exploit, en une image : trois verrous et un rédacteur 5
Ce que l'agent sait faire, et ce qu'il ne sait pas 5
La carte du parcours 6
La règle des trois couches 6
Le chemin complet, en une ligne 6
Trois durées possibles 6
Pourquoi 10 étapes, et pas un seul fichier 7
Avant de commencer 8
L'installation, une seule fois, avec du wifi 8
Les quatre vérifications du jour J 8
Les trois dossiers qui comptent 8
Les cinq choses qu'on ne fait jamais 8
Le test de cinq minutes, avant les étudiants 8
Le plan B, dès maintenant 9
Les dix étapes 10
Étape 1 · Le cerveau seul 10
Étape 2 · La matière 11
Étape 3 · Les mains 12
Étape 4 · Le visage 12
Étape 5 · Le contrôle 13
Étape 6 · La bibliothèque de code 14
Étape 7 · La mémoire documentaire 15
Étape 8 · Le tri et la preuve 16
Étape 9 · La mesure 17
Étape 10 · L'ouverture 19
Les trois extensions 20
Étape 11 (extension) · L'agent pédagogue TP 20
Étape 12 (extension) · Le lien public 21
Étape 13 (extension) · L'exécutable 21

« On devrait pouvoir poser n'importe quelle question » 23
Ce que la phrase compare sans le dire 23
Le contrat : trois conditions, aucune négociable 23
La démonstration en quatre-vingt-dix secondes 23
Les trois verrous, un seul rédacteur 24
Ce que l'étudiant veut vraiment, et quoi répondre 24
Élargir, sans casser le contrat 24
Les limites, dites honnêtement 24
Et si la question est vraiment hors programme ? 25
La réponse en trente secondes 25
Les démonstrations de contrôle 26
1. Le modèle seul contre l'agent — la démonstration qui retourne la classe 26
2. La garde — le refus qui rassure 26
3. La correction — le modèle n'a pas le dernier mot 26
4. La panne — ce qui marche sans modèle 27
5. Le tunnel — la sécurité en une manipulation 27
6. Le TP — l'agent qui refuse de faire le travail 27
Les quatre questions à poser à chaque pause 27
Le conducteur 2 h 28
Si vous êtes en retard 28
Si vous êtes en avance 28
Après la séance 28
Dépannage express 29
Annexes 30
A. L'arborescence de mon_agent_react 30
B. Toutes les commandes, en une page 30
C. Glossaire 30
D. Checklist J-7 31
E. Où est le code, et ce qui a changé 31
F. La fiche à garder sous les yeux 32

Le pitch et l'exploit

Ce qu'on construit, ce que ce n'est pas, et pourquoi c'est difficile

En soixante secondes

Nous allons construire, sur un ordinateur portable et sans internet, un agent qui répond aux questions d'un cours React, explique le code des étudiants, corrige leurs erreurs, fait passer un quiz, et les guide dans un TP sans jamais leur donner la solution.

Il ne prononce pas une seule réponse qui ne vienne d'un document que nous lui avons fourni. À chaque réponse, il affiche le fichier qui l'a nourri. Quand la réponse n'est pas dans le cours, il refuse — et ce refus est instantané, gratuit, et vérifiable.

Ce n'est pas **un chatbot de plus**. C'est un agent documentaire : un modèle de langage qui ne peut parler que de ce qu'on lui a donné à lire.

L'exploit, en une image : trois verrous et un rédacteur

Pièce	Qui la tient	Ce qu'elle empêche
La garde	Python, avant le modèle	Répondre à une question absente du cours (0 appel, 0 invention).
Le tri (RAG)	Python, entre les documents et le modèle	Envoyer 24 000 caractères au modèle et noyer la vraie réponse.
Les pastilles de source	Python, après la lecture	Dire « selon le cours » sans fichier derrière.
La rédaction	Le modèle	Rien : il reformule ce que Python lui a apporté. C'est le seul rôle qui reste.

L'exploit n'est donc pas que le modèle parle. Un modèle parle déjà, tout seul, et il invente. L'exploit est qu'il parle **sans pouvoir mentir sur ce qui compte** : une date, une note, un extrait de code, une étape de TP.

Ce que l'agent sait faire, et ce qu'il ne sait pas

Capacité	Comment	Preuve montrable
Répondre sur le cours	Lit les DOCX/PDF de cours/, trie les passages utiles, rédige	Pastille avec le nom du fichier
Donner une date, un horaire, un prof	Expression régulière sur le calendrier, zéro modèle	Ligne « Enseignant : … » + source
Donner un snippet React	Banque de 22 codes validés par le professeur	Pastille banque_react.py:clé
Corriger du code	Analyse du code, motifs React, exécution en bac à sable, puis explication	Liste de faits + « EXPLICATION IA »
Noter un quiz	Questions et réponses dans Python	Score 1/2 avec le détail
Guider un TP	Plan extrait du PDF, vérificateur de code, seuil 0,6	Étape débloquée, ligne dans progres.csv
Répondre à tout	—	Il refuse : c'est le contrat, pas une panne

La carte du parcours

Ce que chaque étape ajoute, et ce qui casserait sans elle

Lisez ce tableau de gauche à droite : chaque ligne transforme l'agent construit à la ligne précédente. La colonne « ce que ça change pour l'utilisateur » est celle que vous commentez à voix haute ; la colonne « commande » est celle que les étudiants tapent.

#	Étape	Fichier	Ce que ça ajoute	Ce que ça change pour l'utilisateur	Min
1	Le cerveau	etape1.py	Un modèle local qui parle	Il répond, mais invente	8
2	La matière	etape2.py	cours/react.txt + garde	Il répond juste ou refuse	10
3	Les mains	etape3.py	Outil lire_cours, boucle ReAct	Il lit le cours au lieu de s'en souvenir	12
4	Le visage	etape4.py	Serveur web local	On le montre dans un navigateur	8
5	Le contrôle	etape5_web.py	Quiz corrigé par Python	Il peut interroger sans inventer	10
6	La bibliothèque de code	etape6_web.py	22 snippets + correcteur	Il donne du code validé et explique les fautes	12
7	La mémoire documentaire	etape6_web_doc.py	Lecture DOCX/PDF réelle	Il lit vos vrais documents	10
8	Le tri et la preuve	etape8.py	Routage Python + RAG + pastilles	Il montre pourquoi il répond	13
9	La mesure	etape9_tokens.py, etape9c_eval.py	Tokens et évaluation verrouillée	On comprend ses limites, on mesure	8
10	L'ouverture	etape10_ressources.py	Livre + sites + cache	Il cite un livre et la doc officielle	7

La règle des trois couches

Couche	Rôle	Exemples dans l'atelier	Si elle disparaît
Le modèle	Mettre en mots	llama3.2, reformulation, explication, indices	Plus de langage naturel : l'agent redevient un script
Python	Décider et vérifier	garde, tri, calendrier, quiz, AST, seuil de déblocage	Le modèle invente, et rien n'est vérifiable
Les documents	Dire la vérité	cours/, Tps/, banque_react.py, cache web	Le modèle parle de mémoire : plus de preuve

Les étapes 1 à 4 installent les couches une par une. Les étapes 5 à 7 branchent les documents. L'étape 8 les fait travailler ensemble. Les étapes 9 et 10 mesurent et élargissent, sans jamais retirer un verrou.

Le chemin complet, en une ligne

DE LA QUESTION À LA PREUVE

```
question → Python choisit le chemin → (banque | calendrier | garde | RAG)
         → si RAG : lecture de cours/ → tri des passages → modèle rédige
         → Python extrait les sources → l'utilisateur voit la réponse ET la preuve
```

LE POINT DE BASCULE

Tant que l'agent ne fait que répondre, c'est une démonstration. À partir de l'étape 5, il corrige ; à partir de l'étape 8, il prouve ; à partir de l'extension 11, il refuse de faire le TP à la place de l'étudiant. C'est là qu'il devient pédagogique.

Trois durées possibles

Format	Étapes gardées	Ce qu'on sacrifie	Public
90 minutes	1, 2, 3, 8 + les démonstrations de la section «	5, 6, 9, 10 et extensions	Découverte, enseignants pressés

Format	Étapes gardées	Ce qu'on sacrifie	Public
	Les démonstrations de contrôle »		
2 heures (défaut)	Les 10 étapes, 8 en démonstration guidée	Le TP complet (extension 11 montrée)	Étudiants et enseignants
3 heures	Les 10 étapes + app_tp.py en autonomie	Rien	Atelier pratique avec TP

Pourquoi 10 étapes, et pas un seul fichier

Parce que chaque étape ajoute une pièce qu'on peut montrer seule. Si vous donnez app_final.py d'emblée, les étudiants voient une application qui marche ; ils ne voient pas pourquoi elle marche. Les dix étapes rendent chaque décision visible : le modèle seul, la matière, les mains, le visage, le contrôle, la bibliothèque, le tri, la mesure, l'ouverture.

> **À DIRE À LA FIN, MOT POUR MOT**
>
> 1. Le modèle ne lit pas vos documents : Python lit, trie et lui apporte les passages.
> 2. Le modèle propose, Python décide : notes, horaires et code viennent de fichiers.
> 3. Un agent qui répond à tout sans source est un moteur à vraisemblance, pas un agent de cours.

Avant de commencer

Trente minutes de préparation qui sauvent les deux heures

L'installation, une seule fois, avec du wifi

DANS LE DOSSIER MON_AGENT_REACT

```
cd mon_agent_react
installer.bat
ollama list
python -c "import requests, pypdf, transformers; print('libs OK')"
```

RÉSULTAT ATTENDU

```
NAME                ID              SIZE      MODIFIED
llama3.2:latest     a80c4f17acd5    2.0 GB    ...

libs OK
```

installer.bat vérifie Python, installe les trois bibliothèques et télécharge le modèle. Il ne s'installe qu'une fois. Le jour de l'atelier, il n'y a plus rien à télécharger : c'est tout l'intérêt du local.

Les quatre vérifications du jour J

Vérification	Comment	Attendu
Le moteur tourne	ollama list	llama3.2:latest apparaît
Les bibliothèques sont là	python -c "import requests, pypdf, transformers; print('libs OK')"	libs OK
L'interface est hors-ligne	Couper le wifi, lancer python app_final.py	La page reste stylée (CSS local)
Aucun serveur fantôme	Gestionnaire des tâches, ou taskkill /F /IM python.exe	Aucun python.exe ne traîne

Les trois dossiers qui comptent

Dossier	Ce qu'on y met	Qui le lit
cours/	react.txt, vos DOCX, vos PDF, livres/ et cache_web/	lire_cours() dans les étapes 2 à 8
Tps/	le PDF du TP, plan_tp.json, fiche_A.json, progres.csv	app_tp.py (extension 11)
static/	app.css (9 Ko) et mathjax-tex-svg.js (2 Mo)	les trois applications web, sans CDN

Les cinq choses qu'on ne fait jamais

- **Relancer ollama serve** — s'il tourne déjà : l'erreur « bind » est normale, le service est déjà là.
- **Écouter sur 0.0.0.0** — les applications restent en 127.0.0.1. Pour montrer à distance, on utilise un tunnel sortant.
- **Utiliser localhost** — dans les scripts et les démonstrations : 127.0.0.1 évite les détours IPv6 et proxy.
- **Déposer un PDF pirate** — cours/livres/ accueille votre PDF légal ou un extrait libre de droits, rien d'autre.
- **Compresser .venv** — 817 Mo : utilisez les archives déjà préparées, ou robocopy sans .venv.

Le test de cinq minutes, avant les étudiants

Lancez l'agent complet, posez les quatre questions de la section « Les démonstrations de contrôle », vérifiez que les pastilles s'affichent. Si les quatre passent, l'atelier peut commencer. Si une seule échoue, vous savez exactement où chercher : section « Dépannage ».

LE TEST EXPRESS

```
python app_final.py
-> ouvrir http://127.0.0.1:3000
-> « Que dit la matiere sur Tailwind ? »      (attend : une reponse + pastille .docx)
-> « Quand le cours 420-TT4-AS ? »           (attend : lundi 08h-12h, Mourad Sehboub)
-> « donne le code usereducer »              (attend : snippet de la banque)
-> « Explique useReducer ? »                 (attend : Hors-matiere)
```

Le plan B, dès maintenant

SI UNE ÉTAPE CASSE PENDANT LA SÉANCE

Ne réparez pas devant le groupe. Basculez sur app_final.py (l'agent complet, BUILD v9) et rejouez les démonstrations de la section « Les démonstrations de contrôle ». L'atelier survit à une étape ratée ; il ne survit pas à dix minutes de débogage silencieux.

Les dix étapes

Une commande, un résultat attendu, une raison d'être

Chaque étape suit le même gabarit. Les étudiants tapent la commande ; vous lisez à voix haute l'encadré « pourquoi » et la question de contrôle. Si le résultat attendu n'apparaît pas, ne corrigez pas le code : passez au plan B en bas de l'étape, puis à la section « Dépannage ».

Étape 1 · Le cerveau seul

Un modèle local qui parle — et qui invente

FICHIER	DURÉE	MODE	SORTIE
etape1.py	**8 min**	**Console**	**Réponse en 2 à 8 s**

Entendre le moteur. Un modèle de langage tourne sur le portable, sans internet. C'est la voix de l'agent. C'est aussi la démonstration de ce qu'il ne faut pas faire : répondre de mémoire.

Ce que vous faites

- Se placer dans mon_agent_react.
- Poser une question facile : « Explique useState ».
- Poser deux questions pièges : l'horaire du cours 420-TT4-AS, puis le nom de l'enseignant.
- Noter les réponses au tableau : elles sont fluides, plausibles, et fausses.

COMMANDE

```
python etape1.py
```

RÉSULTAT ATTENDU

```
Question React [defaut: Explique useState avec exemple simple] : Explique useState

--- reponse ---

useState est un hook React qui permet de garder un etat...
(sortie variable, 2 a 8 s)

Question React [defaut: Explique useState avec exemple simple] : Quand a lieu le cours 420-TT4-AS ?

--- reponse ---

Le cours a lieu le mardi de 13 h a 16 h avec M. Dupont.
(plausible – et faux : ce n'est pas votre calendrier)
```

POURQUOI CETTE ÉTAPE COMPTE

C'est la couche « modèle » : sans elle, pas de reformulation, pas d'explication, pas d'indice. Mais seule, elle répond avec sa mémoire statistique. Elle ne sait pas ce qu'elle ne sait pas, et elle ne dit jamais « je n'ai pas cette information ». C'est exactement le problème que les neuf étapes suivantes vont encadrer.

Le lien avec les autres étapes

Ce que cette étape rend possible : Un agent qui parle et qu'on peut montrer en dix secondes.

Ce que la suivante ajoute : La matière : le modèle perd sa mémoire et gagne un classeur de 515 caractères.

PIÈGE

Confondre fluidité et vérité. La réponse la plus convaincante de la séance est souvent la plus fausse. Le modèle ne signale pas son incertitude : c'est à Python de le faire.

QUESTION DE CONTRÔLE

« D'où vient l'horaire qu'il vient de donner ? » Réponse attendue : de nulle part. Aucun fichier n'a été lu.

Si ça rate : *Ollama ne répond pas → lancer Ollama, puis vérifier avec ollama list. Modèle absent → ollama pull llama3.2:latest.*

Étape 2 · La matière

Un classeur fini, et un portier qui refuse le reste

FICHIER	DURÉE	MODE	SORTIE
`etape2.py`	**10 min**	**Console**	**Réponse juste ou refus**

Remplacer la mémoire par un document. Le cours React tient dans 515 caractères ; le modèle ne répond plus que là-dessus. Une garde Python refuse les questions dont aucun mot n'apparaît dans le cours, sans même appeler le modèle.

Ce que vous faites

- Ouvrir cours/react.txt et montrer les quatre notions : useState, useEffect, Props, Composant.
- Lancer le script, poser « Explique useState » : la réponse est juste.
- Poser « Explique useReducer » : le refus est immédiat, sans appel au modèle.
- Faire remarquer le compteur : « Cours charge : 515 caracteres ».

COMMANDE

```
python etape2.py
```

RÉSULTAT ATTENDU

```
Cours charge : 515 caracteres depuis cours\react.txt

Question React (vide=quitter) : Explique useState

--- reponse ---

useState garde une valeur qui fait re-afficher le composant...

Question React (vide=quitter) : Explique useReducer

--- reponse ---

Hors-matiere - voir manuel. (bloque par garde Python, 0 appel LLM)
```

> **POURQUOI CETTE ÉTAPE COMPTE**
>
> La matière devient finie, donc vérifiable. Le refus est une fonctionnalité : zéro appel, zéro coût, zéro invention, réponse instantanée. C'est la première fois de la séance qu'un étudiant voit un agent qui admet ne pas savoir — et c'est cela qui rend l'agent crédible.

Le lien avec les autres étapes

Ce que cette étape rend possible : La règle « la réponse doit venir du cours » et le premier verrou.

Ce que la suivante ajoute : Les mains : le classeur devient un outil que le modèle peut demander, au lieu d'être collé dans son prompt.

> **PIÈGE**
>
> La garde est un filtre grossier : un seul mot présent par hasard suffit à laisser passer une question. Elle complète le tri des passages, elle ne le remplace pas. Ne lui demandez pas d'être intelligente ; demandez-lui d'être prévisible.

> **QUESTION DE CONTRÔLE**
>
> « Pourquoi refuse-t-il useReducer alors que llama3.2 sait très bien l'expliquer ? » Réponse attendue : parce que ce chapitre n'est pas dans le cours, et qu'on préfère un refus à une réponse invérifiable.

Si ça rate : *La garde refuse une question légitime ? Ajoutez le document ou le chapitre, pas une exception dans le code.*

Étape 3 · Les mains

Le modèle ne lit pas le disque : il demande, Python exécute

FICHIER	DURÉE	MODE	SORTIE
`etape3.py`	**12 min**	**Console**	**Ligne [outil] + réponse**

Séparer celui qui parle de celui qui lit. Le modèle émet une demande d'outil ; Python ouvre le fichier et renvoie le texte ; le modèle rédige à partir de ce texte. C'est la boucle ReAct, en cinq tours maximum.

Ce que vous faites

- Lancer le script et poser « Que dit le cours sur Props ? ».
- Faire observer la ligne [outil] lire_cours() : c'est Python qui a lu.
- Expliquer les trois temps : le modèle demande, Python exécute, le modèle rédige.
- Montrer le plafond de cinq tours : un agent doit pouvoir s'arrêter.

COMMANDE

```
python etape3.py
```

RÉSULTAT ATTENDU

```
Question React (vide=quitter) : Que dit le cours sur Props ?

--- agent ---

[outil] lire_cours()
Props : donnees du parent vers enfant. Ex: <Bouton titre="Valider" />.
```

POURQUOI CETTE ÉTAPE COMPTE

C'est la séparation des pouvoirs. Le modèle ne peut lire que ce que Python accepte de lui donner : on peut journaliser, limiter, remplacer l'outil, ou couper l'accès. Sans cette boucle, aucune pastille de source n'est possible : on ne peut pas prouver ce qu'on n'a pas tracé.

Le lien avec les autres étapes

Ce que cette étape rend possible : Un agent auditable : chaque lecture passe par une fonction que vous contrôlez.

Ce que la suivante ajoute : Le visage : la même boucle, mais derrière un navigateur.

PIÈGE

llama3.2 écrit parfois l'appel d'outil en texte au lieu de tool_calls ; d'où le parse_secours. Autre erreur classique : renvoyer le résultat sans role=tool ni le même identifiant, ce qui fait boucler le modèle.

QUESTION DE CONTRÔLE

« Qui a ouvert cours/react.txt ? » Réponse attendue : Python, pas le modèle. « Le modèle a-t-il vu tout le fichier ? » Non : seulement les 4 000 premiers caractères, et bientôt seulement les passages utiles.

Si ça rate : *Si le modèle n'appelle jamais l'outil, vérifier que le champ tools est bien envoyé et que la température est à 0.*

Étape 4 · Le visage

Un navigateur, zéro installation côté étudiant

FICHIER	DURÉE	MODE	SORTIE
`etape4.py`	**8 min**	**Navigateur**	**Page servie en local**

Sortir de la console. Python sert une page HTML sur 127.0.0.1:3000 ; l'étudiant tape sa question dans un champ et lit la réponse. Rien à installer sur les postes : un navigateur suffit.

Ce que vous faites

- Lancer le script.
- Ouvrir http://127.0.0.1:3000 dans le navigateur.
- Poser une question sur Props et vérifier que la réponse s'affiche dans la page.
- Faire remarquer l'adresse : 127.0.0.1, donc uniquement cette machine.

COMMANDE

```
python etape4.py
```

RÉSULTAT ATTENDU

```
Ouvrez http://127.0.0.1:3000

(page : « Agent React - TechWeek (local) », un champ, un bouton OK,
 puis la reponse dans un cadre gris)
```

> **POURQUOI CETTE ÉTAPE COMPTE**
>
> Une démonstration qu'on peut projeter, et une frontière de sécurité : le serveur écoute sur 127.0.0.1, pas sur le réseau. On n'ouvre pas de port sur le routeur. Pour montrer à distance, on utilisera un tunnel sortant (extension 12), jamais une redirection de port.

Le lien avec les autres étapes

Ce que cette étape rend possible : Le support de toutes les démonstrations suivantes : boutons, modes, pastilles.

Ce que la suivante ajoute : Le contrôle : des boutons qui déclenchent du code Python, pas des réponses du modèle.

> **PIÈGE**
>
> localhost n'est pas toujours 127.0.0.1 (IPv6, proxy d'entreprise). Un ancien python.exe peut garder le port 3000 : tuez-le avant de relancer. Et ne mettez jamais 0.0.0.0 « pour tester ».

> **QUESTION DE CONTRÔLE**
>
> « Pourquoi un autre étudiant ne voit-il pas cette page ? » Réponse attendue : le serveur n'écoute que sur la boucle locale de cette machine.

Si ça rate : *Port occupé → taskkill /F /IM python.exe, puis relancer. Page blanche → vérifier l'adresse exacte et le BUILD affiché en console.*

Étape 5 · Le contrôle

Le premier geste que le modèle ne fait pas : noter

FICHIER	DURÉE	MODE	SORTIE
etape5_web.py	**10 min**	**Navigateur**	**Score détaillé**

Ajouter un quiz dont les questions, les choix et la bonne réponse vivent dans Python. Le modèle n'écrit rien, ne corrige rien : il n'a même pas la clé. L'agent devient un outil de classe, pas un générateur de QCM.

Ce que vous faites

- Cliquer sur « Lancer Quiz » : deux questions tirées parmi quatre.
- Répondre volontairement faux à une question, puis cliquer sur « Corriger ».
- Lire le score détaillé : 1:B=OK 2:A=NON->B.
- Relancer le quiz : le tirage change, la correction reste exacte.

COMMANDE

```
python etape5_web.py
```

RÉSULTAT ATTENDU

```
Ouvrez http://127.0.0.1:3000

(page : « Agent React - Etape 5 Web (local) », boutons Cours et Lancer Quiz)

[outil] verifier_reponse() [Python]
Score 1/2 - 1:B=OK 2:A=NON->B
```

POURQUOI CETTE ÉTAPE COMPTE

La note ne peut pas être inventée : elle est calculée par du code, deux fois de suite, avec le même résultat. Le modèle n'a aucun rôle. C'est reproductible, gratuit, instantané, et surtout explicable à un étudiant qui conteste sa note : on lui montre la ligne qui a produit le score.

Le lien avec les autres étapes

Ce que cette étape rend possible : Le schéma « bouton → Python → réponse » que tout le reste réutilise.

Ce que la suivante ajoute : La bibliothèque de code : un autre bouton, un autre outil Python, la même logique.

PIÈGE

L'état du quiz vit en mémoire : deux onglets se marchent dessus. En classe, un binôme par navigateur, ou un Reset entre deux passages. Le quiz n'est pas une évaluation notée à distance.

QUESTION DE CONTRÔLE

« Qui a corrigé ? » Réponse attendue : Python. « L'IA peut-elle donner la réponse si on la lui demande ? » Non : elle ne la possède pas.

Si ça rate : *Si le score ne s'affiche pas, vérifier que le bouton Corriger est bien envoyé après le quiz, et qu'aucun pare-feu local ne bloque le POST.*

Étape 6 · La bibliothèque de code

Le code vient du professeur, l'explication vient du modèle

FICHIER	DURÉE	MODE	SORTIE
`etape6_web.py`	**12 min**	**Navigateur**	**Snippet + correction**

Deux régimes qui ne se mélangent jamais. Quand on demande du code, il sort d'une banque de 22 snippets validés par le professeur. Quand on colle du code fautif, Python relève les faits (motifs React, analyse du code, exécution en bac à sable) et le modèle se contente de les expliquer.

Ce que vous faites

- Cliquer sur un bouton de snippet, ou taper « donne le code usereducer ».
- Coller un composant fautif : function carte(){ count=count+1; ... }.
- Lire d'abord la liste CORRECTION PYTHON, puis seulement EXPLICATION IA.
- Faire ajouter un snippet par un étudiant : une entrée dans la banque, et il est disponible.

COMMANDE

```
python etape6_web.py
```

RÉSULTAT ATTENDU

```
[outil] banque_exemples() -> useReducer (etats complexes)
import { useReducer } from 'react';
...

[outil] corriger_code() [Python] + [IA explique]
CORRECTION PYTHON (regles du cours) :
- Ligne count= : interdit par le cours. Utilisez setCount(count+1).
- Composant en minuscule : mettez majuscule ex: function Carte().
- Erreur syntaxe ligne 1 : invalid syntax

EXPLICATION IA (verrouillee sur faits) :
Ta fonction doit s'appeler Carte et modifier l'etat avec setCount...
```

> **POURQUOI CETTE ÉTAPE COMPTE**
>
> Un snippet inventé compile parfois, mais il n'est pas dans votre progression : la banque garantit le niveau, le style et les notions déjà vues. Le correcteur, lui, ne demande pas au modèle de trouver les fautes : il les trouve, puis lui demande de les expliquer. C'est la première application complète de « Prompt = politesse, Code = police ».

Le lien avec les autres étapes

Ce que cette étape rend possible : Une bibliothèque de code fiable et une correction qui ne dépend pas de l'humeur du modèle.

Ce que la suivante ajoute : La mémoire documentaire : les documents du cours remplacent le fichier texte écrit à la main.

> **PIÈGE**
>
> Python ne comprend pas le JSX : l'erreur « invalid syntax » sur un composant est attendue, elle signifie « ce n'est pas du Python », pas « votre React est faux ». Les vraies erreurs React sont les motifs (minuscule, count=). Ajouter un snippet mal nommé le rend introuvable.

> **QUESTION DE CONTRÔLE**
>
> « Qui a écrit le code affiché ? » Réponse attendue : le professeur, dans banque_react.py. « Qui l'a expliqué ? » Le modèle, à partir des faits Python.

Si ça rate : *Si le snippet n'apparaît pas, vérifier la clé dans banque_react.py et la question exacte (« donne le code … »).*

Étape 7 · La mémoire documentaire

Vos vrais documents remplacent le fichier texte

FICHIER	DURÉE	MODE	SORTIE
etape6_web_doc.py	**10 min**	**Navigateur**	**Réponse + nom du fichier**

Passer du .txt écrit à la main aux documents réels : le plan de cours, le calendrier, le guide React, les tendances technologiques. C'est le moment où l'agent devient le vôtre. Le RAG naît ici, par nécessité : quatre documents font 24 000 caractères, trop pour être envoyés à chaque question.

Ce que vous faites

- Déposer un DOCX ou un PDF dans cours/ (le calendrier y est déjà).
- Relancer le script et poser une question sur un passage précis du document.
- Vérifier que la réponse cite le nom du fichier entre crochets.
- Déposer un PDF scanné : l'agent l'ignore et le dit, au lieu d'inventer.

COMMANDE

```
python etape6_web_doc.py
```

RÉSULTAT ATTENDU

```
BUILD porte3-calendrier v2 - Ouvrez http://127.0.0.1:3000

(page : Agents React, champ de question, boutons Quiz et Corriger)

[outil] lire_cours() [calendrier Python, 0 LLM]
D'apres le calendrier :
Cours : 420-TT4-AS
Enseignant : Mourad Sehboub
Horaires : Lundi 08h-12h (À distance)
```

POURQUOI CETTE ÉTAPE COMPTE

C'est le passage à l'échelle : l'agent lit des dizaines de pages sans qu'on réécrive son prompt. Il lit aussi les PDF, ce qui permet d'utiliser le calendrier officiel et le TP. Et il sait dire « ce PDF est une image, je ne peux pas le lire » : une limite avouée vaut mieux qu'une réponse inventée.

Le lien avec les autres étapes

Ce que cette étape rend possible : Un agent qui parle de votre cours, avec vos mots et vos dates.

Ce que la suivante ajoute : Le tri et la preuve : parmi 24 000 caractères, quels passages envoyer, et comment le prouver.

PIÈGE

Un PDF scanné est une image : sans OCR, il ne contient pas de texte. Un .doc renommé en .docx ne s'ouvre pas. Le corpus est coupé à 24 000 caractères : au-delà, les derniers documents disparaissent silencieusement.

QUESTION DE CONTRÔLE

« D'où vient la réponse ? » Réponse attendue : du fichier cité, pas de la mémoire du modèle.

Si ça rate : *Si un document n'apparaît jamais, vérifier son extension réelle et sa taille ; réduire le corpus à l'essentiel.*

Étape 8 · Le tri et la preuve

Python choisit le chemin, le modèle rédige, la source s'affiche

FICHIER	DURÉE	MODE	SORTIE
etape8.py	**13 min**	**Console**	**5 questions, 1 seul appel LLM**

Assembler les pièces. Avant d'appeler le modèle, Python classe la question : demande de code, date, hors-sujet, ou question documentaire. Puis il ne lui donne que les passages utiles. Enfin, il extrait les sources du texte qu'il a réellement envoyé — ce ne sont pas des remerciements, ce sont des preuves.

Ce que vous faites

- Lancer la démonstration : cinq questions, cinq chemins différents.
- Compter les appels au modèle : un seul sur les cinq.
- Montrer les pastilles de source, et le mode dégradé quand Ollama est absent.
- Relancer en interactif et laisser un étudiant poser sa question.

COMMANDE

```
python etape8.py --demo
python etape8.py
```

RÉSULTAT ATTENDU

```
etape8 v1 - hybride + RAG + preuves
Matiere chargee : 19278 caracteres depuis cours/  |  Ollama : OK

QUESTION : Que dit la matiere sur Tailwind ?
[outil] lire_cours() / tri Python | chemin : RAG (Python trie, LLM redige) | 1 appel(s) LLM
Selon le guide, Tailwind CSS propose des classes utilitaires...
Sources consultees (preuve Python) : [Guide_complet_React_TypeScript_Tailwind.docx]

QUESTION : Quand le cours 420-TT4-AS ?
[outil] lire_cours() / tri Python | chemin : CALENDRIER (Python seul) | 0 appel(s) LLM
Cours : 420-TT4-AS
Enseignant : Mourad Sehboub
Horaires : Lundi 08h-12h (À distance)
Sources consultees (preuve Python) : [TendancesTechnologiques_420-TT4-AS.docx]
[Tendances...Calendrier...pdf]

QUESTION : Explique useReducer ?
chemin : GARDE (Python seul) | 0 appel(s) LLM
Hors-matiere - voir manuel. Aucun mot de la question n'apparait dans cours/.

QUESTION : donne le code usereducer
chemin : BANQUE (Python seul) | 0 appel(s) LLM
Banque (validee par le prof) : useReducer (etats complexes)

QUESTION : Quelle est la recette de la tarte aux pommes ?
chemin : GARDE (Python seul) | 0 appel(s) LLM
Hors-matiere - voir manuel.
```

POURQUOI CETTE ÉTAPE COMPTE

C'est le cœur de l'exploit, et il tient en cinq observations. 1) Le chemin est décidé par du code, donc il est prévisible et explicable. 2) Quatre questions sur cinq ne coûtent aucun appel au modèle : elles sont instantanées et gratuites. 3) Les sources sont extraites du texte envoyé, pas déclarées par le modèle : une preuve, pas une politesse. 4) Le RAG n'envoie que 3 500 caractères au lieu de 24 000 : la fenêtre de contexte est respectée. 5) Ollama éteint, l'agent ne mens pas : il montre les passages et annonce le mode dégradé.

Le lien avec les autres étapes

Ce que cette étape rend possible : Le produit complet : app_final.py n'est que cette étape, avec l'interface et tous les modes (BUILD v9).

Ce que la suivante ajoute : La mesure : pourquoi 3 500 caractères et pas tout le cours (tokens), et comment évaluer sans tricher.

PIÈGE

La garde est volontairement grossière : « Explique useReducer » est refusé même si le modèle sait. C'est le contrat. Le tri par mots-clés peut choisir de mauvais passages sur une question abstraite : on améliore en ajoutant des documents, jamais en supprimant la preuve. Et on n'envoie jamais tout le corpus : 24 000 caractères, c'est déjà les trois quarts de la fenêtre du modèle.

QUESTION DE CONTRÔLE

« Combien de fois le modèle a-t-il été appelé pour cinq questions ? » Réponse attendue : une. « Qui a produit les pastilles ? » Python, à partir du texte lu.

Si ça rate : *etape8.py est une démonstration, pas le produit : si elle échoue, lancez app_final.py, qui contient la même logique.*

Étape 9 · La mesure

Pourquoi on ne peut pas tout donner, et comment évaluer sans tricher

FICHIER	DURÉE	MODE	SORTIE
etape9_tokens.py, etape9c_eval.py	**8 min**	**Console**	**Tokens + score CSV**

Comprendre la contrainte qui justifie tout le reste. Un modèle ne voit pas des caractères, il voit des tokens ; sa fenêtre est finie. Et mesurer ce que les étudiants ont compris avec un QCM dont les bonnes réponses sont verrouillées par Python.

Ce que vous faites

- Lancer la tokenisation sur trois phrases (du code, une question, une date).
- Faire le lien avec la limite de 8 192 tokens et la taille des passages de l'étape 8.
- Lancer l'évaluation, répondre aux questions, lire le score.
- Montrer le fichier resultats/eval.csv : une ligne par passage, pas une note inventée.

COMMANDE

```
python etape9_tokens.py
python etape9c_eval.py
```

RÉSULTAT ATTENDU

```
'const [count, setCount] = useState(0);'
  -> 13 tokens : [9979, 685, 9127, 11, 900, 12332, ...]...
  -> decode : 'const [count, setCount] ='

'Quand le cours TendancesTechnologiques ?'
  -> 11 tokens : [4507, 392, 443, 1093, ...]...

'Lundi 08h-12h avec Mourad Sehboub'
  -> 16 tokens : [43, 917, 72, 8487, ...]...

Lien atelier : NUM_CTX=8192 tokens max, vos 12000 car. cours sont tronques.
D'ou passages_utiles().

--- evaluation ---
12 phrases candidates dans cours/

1. [IA auteur] Pourquoi utilise-t-on useState dans un composant ?
   A) ...   B) ...   C) ...
Votre lettre : B
OK   [SOURCE cours/ + IA]

Score comprehension : 2/3 -> resultats/eval.csv
(si Ollama est absent : « 1. [Python seul, IA indisponible] ... » puis [SOURCE cours/])
```

POURQUOI CETTE ÉTAPE COMPTE

Le RAG n'est pas une mode : c'est une conséquence arithmétique. Un cours de 12 000 caractères dépasse déjà la fenêtre confortable ; quatre documents la remplissent. Et un QCM généré par le modèle mais verrouillé par Python — la bonne réponse doit se retrouver dans le passage, les leurres doivent en différer — montre qu'on peut utiliser l'IA sans lui donner le pouvoir de noter.

Le lien avec les autres étapes

Ce que cette étape rend possible : La justification chiffrée du tri, et un outil d'évaluation réutilisable pour n'importe quel cours.

Ce que la suivante ajoute : L'ouverture : un livre, des sites officiels, et un cache pour que la démonstration survive au wifi du collège.

PIÈGE

transformers télécharge le tokenizer gpt2 (environ 500 Ko) : à faire une fois, avec wifi, avant l'atelier. torch n'est nécessaire que pour l'option sentiment (etape9b), pas pour les tokens.

QUESTION DE CONTRÔLE

« Pourquoi ne pas envoyer les quatre documents d'un coup ? » Réponse attendue : 8 192 tokens, et il faut encore de la place pour la réponse.

Si ça rate : *Pas de wifi pour le tokenizer → l'étape 9 est optionnelle, passez à la 10.*

Étape 10 · L'ouverture

Un livre, des sites officiels, et un cache pour le hors-ligne

FICHIER	DURÉE	MODE	SORTIE
`etape10_ressources.py`	7 min	Console	Liste livre + sites

Élargir la bibliothèque sans casser le contrat. Un PDF légal dans cours/livres/, des sites de référence, et un cache local : la première visite télécharge, les suivantes lisent le cache. La preuve indique (live) ou (cache).

Ce que vous faites

- Lancer le script et lire la liste : livre attendu, chapitres reliés à la banque, sites.
- Déposer votre PDF légal dans cours/livres/, relancer, vérifier qu'il est détecté.
- Dans app_final.py, poser une question sur react.dev et observer la pastille (live) puis (cache).
- Rappeler la règle : aucun PDF pirate dans l'atelier.

COMMANDE

```
python etape10_ressources.py
```

RÉSULTAT ATTENDU

```
[Ressources React - livre + web]

LIVRE : React - Apprendre pas a pas (dossier du prof)
Fichier attendu : cours/livres/React_livre.pdf
PDF presents : (aucun - ajoutez-en un)

SITES POPULAIRES :
  - React officiel - react.dev : https://react.dev
```

POURQUOI CETTE ÉTAPE COMPTE

L'agent peut désormais citer un livre et la documentation officielle, pas seulement vos notes. Le cache est ce qui rend la démonstration reproductible : si le wifi du collège tombe, ou si le CDN est bloqué, l'agent répond quand même avec ce qu'il a déjà lu.

Le lien avec les autres étapes

Ce que cette étape rend possible : Une bibliothèque extensible, avec une trace (live) ou (cache) pour chaque source web.

Ce que la suivante ajoute : Les extensions : le même principe appliqué aux TP (app_tp.py), puis au partage et à l'exécutable.

PIÈGE

Un PDF scanné ne passe pas. Le cache ne se rafraîchit que si vous supprimez le fichier correspondant dans cours/cache_web/. Et un site peut changer : la preuve dit (cache), pas « la vérité ». Sans wifi, montrez le mode (cache) : c'est précisément l'intérêt du dispositif.

QUESTION DE CONTRÔLE

« Pourquoi la réponse dit-elle (cache) ? » Réponse attendue : la page a déjà été téléchargée lors d'une démonstration précédente.

Les trois extensions

Partager, faire travailler, livrer — si le temps le permet

Les dix étapes construisent l'agent. Les trois extensions montrent ce qu'on peut en faire en classe, à distance et sur une clé USB. Elles ne sont pas nécessaires pour comprendre l'exploit ; elles le rendent utilisable.

Étape 11 (extension) · L'agent pédagogue TP

Il ne donne pas la solution : il débloque

FICHIER	DURÉE	MODE	SORTIE
app_tp.py	**10 min (démo)**	**Navigateur**	**Port 3002**

Un agent qui lit le PDF du TP, n'en montre qu'une étape à la fois, donne un indice sans jamais donner la solution, et ne passe à l'étape suivante que si le code collé passe le vérificateur Python.

Ce que vous faites

- Lancer app_tp.py : il tourne à côté de app_final.py, sur le port 3002.
- Ouvrir la fiche de l'étape A : but, étapes A1 à A7, squelettes, question de contrôle.
- Coller un code incomplet : le déblocage est refusé, un indice apparaît.
- Coller un code qui passe le seuil : l'étape B se débloque, progres.csv s'allonge.

COMMANDE

```
python app_tp.py
```

RÉSULTAT ATTENDU

```
BUILD app-tp v4 (JS valide) - Ouvrez http://127.0.0.1:3002

(page : plan A-E, fiche de l'etape courante)
A. Configuration de A a Z         60 min
B. Composer une interface         50 min
C. Rendre l'application interactive  40 min
D. Afficher les donnees d'une API 40 min
E. Formulaire et deploiement      30 min
```

POURQUOI CETTE ÉTAPE COMPTE

C'est l'aboutissement pédagogique : le modèle ne fait pas le TP à la place de l'étudiant. Le seuil de 0,6 est dans Python ; les indices sont du texte court ; la décision de débloquer appartient au code. Un agent qui ferait le TP serait un anti-apprentissage : ici, il refuse activement de le faire.

Le lien avec les autres étapes

Ce que cette étape rend possible : Un tuteur qui accompagne sans remplacer, et une trace de progression par binôme.

Ce que la suivante ajoute : Le partage : montrer cet agent à distance, sans ouvrir de port.

PIÈGE

L'état est partagé entre tous les visiteurs : un seul binôme à la fois si vous partagez. Les fiches sont des fichiers JSON éditables : le professeur garde la main sur le contenu. Et avant chaque démonstration, tuez les python.exe fantômes et vérifiez la ligne BUILD.

QUESTION DE CONTRÔLE

« Qui décide de débloquer l'étape ? » Réponse attendue : Python, sur le code collé. « L'IA peut-elle donner la solution ? » Non : ses consignes l'en empêchent, et le vérificateur ne lit que le code.

Si ça rate : *Port 3002 occupé → tuer l'ancien processus. Plan absent → vérifier Tps/plan_tp.json et Tps/fiche_A.json.*

Étape 12 (extension) · Le lien public

Un tunnel sortant, pas un port ouvert

FICHIER	DURÉE	MODE	SORTIE
`partager.py / partager.bat`	**5 min**	**Navigateur distant**	**URL trycloudflare.com**

Montrer l'agent sur un autre ordinateur ou un téléphone, sans compte, sans clé API et sans toucher au routeur. Le PC ouvre un tunnel sortant vers Cloudflare ; le lien public est jetable et meurt avec la fenêtre.

Ce que vous faites

- Double-cliquer sur partager.bat (ou lancer python partager.py).
- Lire l'URL affichée : elle finit par ?k= une clé aléatoire.
- Ouvrir l'URL sans la clé : 403. Avec la clé : la page, puis un cookie.
- Fermer la fenêtre : le lien ne fonctionne plus (erreur 530).

COMMANDE

```
partager.bat
python partager.py 3002        (partage app_tp.py)
python partager.py --sans-cle  (lien ouvert, deconseille)
```

RÉSULTAT ATTENDU

```
https://exemple-aleatoire.trycloudflare.com/?k=6hxJ5ZgS

sans ?k=  -> 403 : cle d acces manquante.
avec ?k=  -> 200 : la page de l'agent, puis cookie de session.
```

POURQUOI CETTE ÉTAPE COMPTE

C'est la réponse au « on ne peut pas montrer à distance » sans faire de trou de sécurité. Le PC se connecte vers l'extérieur en 443 ; Ollama reste en 11434, sur la boucle locale. Aucun compte, aucune clé API, aucun port ouvert sur le routeur.

Le lien avec les autres étapes

Ce que cette étape rend possible : Une démonstration à distance en trente secondes, révocable en fermant la fenêtre.

Ce que la suivante ajoute : L'exécutable : livrer l'agent sur une clé USB, sans Python.

PIÈGE

L'état est partagé : tous les visiteurs voient le même quiz, le même TP, le même progres.csv. Le lien change à chaque lancement et le laptop doit rester allumé. DNS peut mettre 30 secondes ; cloudflared (52 Mo) est retéléchargé s'il manque.

QUESTION DE CONTRÔLE

« Pourquoi n'y a-t-il pas de port ouvert sur le routeur ? » Réponse attendue : le tunnel part du PC vers l'extérieur ; rien n'entre directement.

Si ça rate : *403 → il manque ?k=. DNS name does not exist → attendre 30 s. Le lien meurt → la fenêtre a été fermée.*

Étape 13 (extension) · L'exécutable

Une clé USB qui marche sans Python installé

FICHIER	DURÉE	MODE	SORTIE
`build_exe.bat, dist\AgentReact`	**10 min (hors séance)**	**Windows**	**AgentReact.exe**

Emballer l'application, les documents, le CSS et MathJax dans un dossier exécutable. L'étudiant double-clique, le navigateur s'ouvre ; seuls Ollama et le modèle restent à installer.

Ce que vous faites

- Lancer build_exe.bat sur la machine de préparation (avec wifi, une seule fois).
- Copier dist\AgentReact, installer.bat et le lien ollama.com/download sur la clé USB.
- Sur un poste neuf : installer Ollama, puis lancer AgentReact.exe.
- Vérifier la ligne BUILD en console : c'est elle qui dit quelle version tourne.

COMMANDE

```
build_exe.bat
dist\AgentReact\AgentReact.exe
```

RÉSULTAT ATTENDU

```
BUILD app-finale v9 (multithread) - Ouvrez http://127.0.0.1:3000

(le navigateur s'ouvre sur la page de l'agent)
```

POURQUOI CETTE ÉTAPE COMPTE

C'est la version « je n'ai pas le temps d'installer Python » : Python, les documents et les ressources sont embarqués. Ollama et llama3.2 restent à part, parce qu'un modèle de 2 Go ne se met pas dans un exécutable de cette taille — et parce qu'il doit tourner en service.

Le lien avec les autres étapes

Ce que cette étape rend possible : Une livraison autonome, reproductible, pour les postes sans environnement de développement.

Ce que la suivante ajoute : Fin du parcours : vous avez l'agent, ses preuves, son tuteur de TP et sa version portable.

PIÈGE

On construit en --onedir, pas en --onefile : l'exe a besoin de cours/, static/ et des fichiers Python à côté. torch et transformers sont exclus : l'étape 9 n'est pas dans l'exe. Et l'exe ne contient évidemment pas Ollama.

QUESTION DE CONTRÔLE

« Qu'est-ce qui n'est pas dans l'exe ? » Réponse attendue : Ollama et le modèle, qui s'installent à part.

Si ça rate : *L'exe ne démarre pas → vérifier qu'il est bien dans son dossier, avec cours/ et static/ à côté.*

« On devrait pouvoir poser n'importe quelle question »

La remarque des étudiants, prise au sérieux — et la réponse en trente secondes

C'est la question la plus utile de l'atelier. Elle mérite mieux qu'un slogan. Elle révèle une confusion entre deux objets qui parlent de la même façon, et qui ne promettent pas la même chose.

Ce que la phrase compare sans le dire

	Le chatbot général (étape 1)	L'agent pédagogique (étape 8)
Questions acceptées	Toutes	Toutes celles dont la réponse est dans les documents
Source de la réponse	Sa mémoire d'entraînement	Un fichier, un passage, une ligne
Vérifiable	Non	Oui, en cinq secondes
Quand il ne sait pas	Il invente, avec assurance	Il refuse, ou il cite
Peut noter un quiz	Non	Oui, le score est calculé
Utile en cours	Pour discuter	Pour répondre, corriger, guider
Coût d'une erreur	Une phrase fausse	Une note fausse, une date fausse

Le premier est impressionnant. Le second est utilisable. L'atelier construit le second, mais commence par le premier : c'est l'étape 1, et c'est volontaire. On ne peut pas comprendre le contrat de l'agent si on n'a pas vu, en direct, le modèle inventer un horaire.

Le contrat : trois conditions, aucune négociable

- **La réponse est dans la matière** — documents de cours/, banque de code validée, calendrier, ou cache web. Pas dans la mémoire du modèle.
- **La question est reconnaissable** — au moins un mot important doit exister dans les documents. Sinon la garde refuse, et le dit.
- **L'agent peut dire « je ne sais pas »** — et ce refus est une réponse de première classe : instantané, gratuit, vérifiable.

Ces trois conditions ne limitent pas l'agent : elles le rendent citable. Un agent qui répond à tout sans source ne peut pas être utilisé pour une note, un horaire ou un extrait de code. Il peut seulement être utilisé pour discuter.

La démonstration en quatre-vingt-dix secondes

C'est la batteries de questions à faire devant la classe, dans cet ordre. Elle est déjà scriptée dans etape8.py --demo ; vous pouvez aussi la jouer dans app_final.py.

Question	Chemin	LLM	Preuve affichée	Ce que ça prouve
Que dit la matière sur Tailwind ?	RAG	1	Le nom du .docx	Il lit et il cite
Quand le cours 420-TT4-AS ?	Calendrier	0	Le fichier du calendrier	La date ne sort pas du modèle
Donne le code useReducer	Banque	0	banque_react.py:usereducer	Le code ne sort pas du modèle
Explique useReducer ?	Garde	0	aucune	Il refuse ce qu'il n'a pas lu
Quelle est la recette de la tarte aux pommes ?	Garde	0	aucune	Le hors-sujet est instantané

> **LA PHRASE À DIRE APRÈS LA DÉMONSTRATION**
>
> « Sur cinq questions, le modèle n'a été appelé qu'une fois. Les quatre autres réponses sont du code, des fichiers, et zéro invention. Ce n'est pas un chatbot plus faible : c'est un chatbot qui a des comptes à rendre. »

Les trois verrous, un seul rédacteur

Verrou	Ce qu'il bloque	Où le voir
La garde	Une question dont aucun mot n'est dans le cours	etape2.py, etape8.py
Le tri (RAG)	L'envoi de 24 000 caractères et la noyade du passage utile	passages_utiles(), etape8.py
Les pastilles	L'affirmation « selon le cours » sans fichier derrière	extraire_sources(), app_final.py

Le rédacteur, lui, n'a qu'un rôle : mettre en français ce que Python lui a apporté. C'est pour cela qu'on peut le remplacer par un autre modèle sans changer le contrat. Le jour où un modèle plus petit, plus rapide ou plus français arrive, on le branche : les verrous, eux, ne bougent pas.

Ce que l'étudiant veut vraiment, et quoi répondre

Ce qu'il dit	Ce qu'il cherche	Votre réponse
« Je devrais pouvoir demander n'importe quoi »	Tester les limites, comme avec ChatGPT	« Tu peux : demande n'importe quoi sur le cours. Pour le reste, l'agent te dira qu'il ne sait pas — et c'est ce qui rend ses réponses fiables. »
« Donne-moi la réponse du quiz »	Contourner l'exercice	« Il ne l'a pas : les réponses sont dans Python, pas dans le modèle. »
« Il refuse useReducer, il est nul »	Comprendre pourquoi	« Ajoute le chapitre useReducer au cours, relance, et il répondra. Le refus n'est pas une limite du modèle : c'est une limite de ta matière. »
« Pourquoi pas ChatGPT ? »	Comprendre l'intérêt du local	« ChatGPT ne lit pas ton cours, ne cite pas tes fichiers, et ne peut pas être branché sur ta banque de code. Ici, tout est chez toi. »

Élargir, sans casser le contrat

Geste	Effet	Temps
Ajouter un DOCX ou un PDF dans cours/	De nouvelles questions deviennent légitimes	1 min
Ajouter un snippet dans banque_react.py	Un nouveau « donne le code … » fonctionne	2 min
Ajouter un site dans le cache web	Une source officielle devient citable	5 min
Ajuster les mots de la garde	Moins de refus, plus de bruit : à faire avec mesure	10 min
Mesurer avec etape9c_eval.py	Savoir ce qui est compris, pas seulement répondu	10 min

La bonne façon d'élargir n'est jamais de retirer un verrou. C'est d'ajouter de la matière, puis de vérifier que la preuve suit. Un agent qui accepte tout parce qu'on a supprimé la garde n'est pas plus utile : il est seulement plus difficile à contredire.

Les limites, dites honnêtement

- **PDF scannés** — sans OCR, ils ne contiennent pas de texte : l'agent l'annonce et les ignore.
- **Questions à plusieurs documents** — le tri par mots choisit des passages, il ne raisonne pas sur une chaîne de preuves.

- **Questions abstraites** — « quelle est la philosophie de React ? » peut trouver de mauvais passages : le tri est lexical, pas sémantique.
- **Langue et formulation** — une question très éloignée du vocabulaire du cours peut être refusée à tort.
- **Fenêtre de contexte** — 8 192 tokens : au-delà, on ne peut pas tout envoyer, d'où le tri.
- **Garde grossière** — un mot présent par hasard suffit à laisser passer : elle réduit le bruit, elle ne prouve pas la pertinence.

Ces limites ne sont pas des défauts cachés : elles sont la raison d'être des étapes 8 et 9. Un agent qui prétendrait ne pas les avoir serait en train de vous vendre quelque chose.

Et si la question est vraiment hors programme ?

Alors on change d'outil, pas de discours. L'étape 1 est là pour cela : elle répond à tout, avec sa mémoire, et elle ne prétend rien prouver. On peut l'utiliser pour une discussion ouverte, une explication générale, un brainstorming. On ne l'utilise pas pour une note, une date ou un extrait de code.

> **LA DISTINCTION À GARDER EN TÊTE**
>
> Un agent pour apprendre (ouvert, bavard, sans preuve) et un agent pour évaluer (fermé, cité, vérifiable). Les deux sont utiles. Les confondre est la seule vraie erreur.

La réponse en trente secondes

> **À RÉCITER**
>
> « L'agent ne sait pas tout : il sait ce qu'il a lu, et il vous montre où. Si vous voulez qu'il réponde à une nouvelle question, ajoutez le document — et il vous dira de quel passage vient la réponse. C'est moins impressionnant qu'un chatbot qui invente, mais c'est utilisable pour apprendre et pour noter. »

Les démonstrations de contrôle

Six preuves à montrer, et la phrase qui va avec

Une démonstration ne vaut que si elle peut échouer. Ces six-là sont choisies pour cela : chacune montre une décision prise par Python, ou une limite assumée par l'agent. Faites-en trois avant la pause, les trois autres après.

#	Démonstration	Ce que vous faites	Ce que ça prouve	Min
1	Le modèle seul contre l'agent	Poser le même horaire à etape1.py puis à app_final.py	La différence, c'est le fichier, pas le modèle	1
2	La garde et la banque	« Explique useReducer ? » puis « Donne le code useReducer »	Le refus est instantané, le code vient du professeur	1
3	La correction	Coller un composant fautif	Python tranche, le modèle explique	1
4	La panne	Fermer Ollama, relancer app_final.py	Le Python continue de fonctionner	1,5
5	Le tunnel	Ouvrir le lien sans puis avec ?k=	403 puis 200, sans port ouvert	2
6	Le TP verrouillé	Coller un code incomplet dans app_tp.py	L'IA ne fait pas le TP	2

1. Le modèle seul contre l'agent — la démonstration qui retourne la classe

C'est celle qui répond à l'objection avant même qu'elle soit posée. Elle prend une minute et elle est imparable, parce que c'est le même modèle, la même question, et deux résultats.

LE SCRIPT, MOT POUR MOT

```
Etape 1 : « Quand a lieu le cours 420-TT4-AS ? »
  -> le modele repond : mardi 13 h, avec M. Dupont.       (plausible, faux)

Etape 8 / app_final : la meme question
  -> chemin : CALENDRIER (Python seul), 0 appel LLM
  -> Lundi 08h-12h, Mourad Sehboub
  -> Sources : [TendancesTechnologiques_420-TT4-AS.docx] [Calendrier...pdf]
```

Ce qu'il faut dire : « Le modèle n'a pas changé. Ce qui a changé, c'est qu'on ne lui demande plus de se souvenir : on lui donne le calendrier, et c'est Python qui lit la date. »

2. La garde — le refus qui rassure

Poser « Explique useReducer ? » (sans demander le code). L'agent répond « Hors-matiere - voir manuel », en zéro appel. Enchaîner avec « Donne le code useReducer » : la banque répond, avec le snippet validé. Deux questions voisines, deux chemins différents, et une preuve dans le second cas.

Ce qu'il faut dire : « Il ne refuse pas parce qu'il est bête. Il refuse parce que le concept n'est pas dans le cours. Ajoutez le chapitre, relancez, et il répondra. »

3. La correction — le modèle n'a pas le dernier mot

Coller un composant en minuscule avec count=count+1. Lire d'abord la liste CORRECTION PYTHON, puis seulement l'EXPLICATION IA. Insister sur l'ordre : les faits d'abord, la politesse ensuite.

Ce qu'il faut dire : « Si l'IA se trompe dans l'explication, la correction reste juste. C'est ça, la séparation des pouvoirs. »

4. La panne — ce qui marche sans modèle

Fermer Ollama (quitter l'application), puis relancer app_final.py. Le quiz, la banque de codes, le calendrier et le TP continuent de fonctionner. Le mode Cours affiche les passages trouvés et annonce que l'IA est indisponible. Revenir ensuite au mode normal.

Ce qu'il faut dire : « Tout ce qui est vérifiable ne dépend pas du modèle. C'est pour cela qu'on peut l'utiliser en évaluation. »

5. Le tunnel — la sécurité en une manipulation

Lancer partager.bat, ouvrir l'URL sans ?k= (403), puis avec (200). Fermer la fenêtre : le lien meurt. Rappeler qu'aucun port n'a été ouvert sur le routeur.

Ce qu'il faut dire : « Le PC se connecte vers l'extérieur. Rien n'entre directement. Et quand je ferme cette fenêtre, le lien public n'existe plus. »

6. Le TP — l'agent qui refuse de faire le travail

Dans app_tp.py, coller un code incomplet : l'étape ne se débloque pas, un indice apparaît. Coller un code qui passe le seuil : l'étape suivante s'ouvre. Montrer progres.csv.

Ce qu'il faut dire : « L'IA ne peut pas vous donner la solution : elle ne décide même pas du déblocage. C'est le code qui vérifie. »

Les quatre questions à poser à chaque pause

Question	Réponse attendue	Si l'étudiant se trompe
Qui a lu le document ?	Python, via l'outil	Revenir à la ligne [outil] de l'étape 3
Qui a écrit la réponse ?	Le modèle, à partir des passages	Montrer le prompt système
Qui a décidé de refuser ?	Python, la garde	Relancer etape8.py --demo
Qui a corrigé le quiz ?	Python	Montrer le score et le code de correction

> **LA PHRASE DE TRANSITION, À CHAQUE FOIS**
>
> « Ce n'est pas le modèle qui est intelligent, c'est le montage autour de lui. Et ce montage, vous venez de le construire. »

Le conducteur 2 h

Minute par minute, avec ce que vous dites

Les durées incluent la manipulation par les étudiants. Si un groupe est plus lent, coupez court aux étapes 9 et 10 : elles illustrent, elles ne conditionnent pas la compréhension. Ne coupez jamais l'étape 8 : c'est elle qui porte l'exploit.

Min	Ce que vous faites	Ce que vous dites	Étape
0-5	Accueil, vérification d'Ollama, ouverture du dossier	« Aujourd'hui, on ne va pas utiliser une IA : on va la construire, et l'empêcher de mentir. »	—
5-13	Étape 1, trois questions dont deux pièges	« Retenez la réponse à l'horaire : elle est fausse. On va voir pourquoi. »	1
13-23	Étape 2, le cours et le refus	« Le refus est une fonctionnalité, pas un bug. »	2
23-35	Étape 3, la boucle et la ligne [outil]	« Le modèle ne lit pas le disque. C'est Python qui lit. »	3
35-43	Étape 4, la page dans le navigateur	« 127.0.0.1 : cette page n'existe que sur ce PC. »	4
43-53	Étape 5, quiz et score détaillé	« La note est calculée. L'IA n'a pas la clé. »	5
53-65	Étape 6, banque et correction	« Le code vient du prof, l'explication vient du modèle. »	6
65-75	Étape 7, vos vrais documents	« Maintenant il lit votre calendrier, pas le mien. »	7
75-88	Étape 8, les cinq chemins	« Une seule question sur cinq a besoin du modèle. »	8
88-96	Étape 9, tokens et évaluation	« Voilà pourquoi on ne peut pas tout donner. »	9
96-103	Étape 10, livre et cache	« Le wifi tombe ? Le cache répond. »	10
103-112	Extension 11, le TP verrouillé (démo)	« L'IA refuse de faire le TP à votre place. »	11
112-120	Objection, trois phrases, synthèse	« L'agent ne sait pas tout : il sait ce qu'il a lu, et il montre où. »	—

Si vous êtes en retard

- Sautez les étapes 9 et 10 : elles seront lues sur le document.
- Gardez 1, 2, 3, 8 et les démonstrations de la section précédente. C'est le squelette de l'argument.
- Ne sautez pas l'étape 2 : sans le refus, l'étape 8 n'a pas de sens.

Si vous êtes en avance

- Faites ajouter un document par un étudiant, puis reposez la question qui était refusée.
- Faites ajouter un snippet à la banque, puis demandez-le.
- Lancez app_tp.py en autonomie sur 15 minutes et circulez.

Après la séance

À faire	Pourquoi
Fermer les tunnels et tuer les python.exe	Un lien public oublié reste ouvert
Sauvegarder le dossier (sans .venv)	5,8 Mo suffisent ; .venv fait 817 Mo
Noter les questions restées sans réponse	Elles deviennent les documents de la prochaine séance
Relancer etape9c_eval.py	Mesurer ce qui est compris, pas seulement ce qui a été montré

Dépannage express

Les trois commandes de survie, puis les cas courants

À ESSAYER DANS CET ORDRE

```
taskkill /F /IM python.exe     (tue les serveurs fantomes)
ollama list                    (le moteur et le modele sont-ils la ?)
python app_final.py            (relance l'agent complet, BUILD v9)
```

Symptôme	Cause probable	Ce que vous faites
Connection refused	Ollama n'est pas lancé	Démarrer Ollama ; sinon ollama serve
model not found	llama3.2 absent	ollama pull llama3.2:latest
Le port 3000 est occupé	Un ancien python.exe tourne	taskkill /F /IM python.exe, puis relancer
Page blanche	Mauvaise adresse ou mauvais BUILD	Ouvrir http://127.0.0.1:3000, vérifier la ligne BUILD en console
Page sans style	static/ absent ou CDN utilisé	Vérifier static/app.css ; aucune page ne doit appeler un CDN
Accents bizarres dans la console	Encodage PowerShell	set PYTHONIOENCODING=utf-8 ; les fichiers sont corrects
403 sur le lien public	Il manque ?k= ou le cookie a été effacé	Rouvrir l'URL complète avec la clé
DNS name does not exist	Le tunnel vient de démarrer	Attendre 30 secondes
530 sur le lien public	La fenêtre du tunnel est fermée	Relancer partager.bat
PDF ignoré	PDF scanné (image)	Utiliser un PDF texte, ou faire de l'OCR
DOCX illisible	Fichier .doc renommé	Enregistrer vraiment en .docx
« Clique d'abord Lancer Quiz »	État du quiz perdu	Relancer le quiz, un binôme par navigateur
Le TP ne débloque pas	Code sous le seuil 0,6	Lire l'indice, compléter, recoller
ImportError transformers	Conflit de versions	pip install -r requirements.txt (le verrou huggingface-hub<2 est inclus)
Première réponse très lente	Le modèle se charge en mémoire	Attendre 10 à 20 s, les suivantes sont rapides
L'exe ne démarre pas	Il a été déplacé hors de son dossier	Garder dist\AgentReact entier, avec cours/ et static/

LA RÈGLE D'OR DU DÉPANNAGE EN SÉANCE

Deux minutes maximum. Au-delà, vous basculez sur app_final.py et vous continuez. Un atelier qui avance avec une démo partielle vaut mieux qu'un atelier qui s'arrête sur un port occupé.

Annexes

L'arborescence, toutes les commandes, le glossaire, la checklist

A. L'arborescence de mon_agent_react

LES FICHIERS QUE VOUS MANIPULEZ

```
mon_agent_react\
├─ etape1.py … etape10_ressources.py   les dix etapes (console ou web)
├─ etape8.py                           l'etape manquante, reconstruite et testee
├─ etape9b_sentiment.py, etape9c_eval.py  options de mesure
├─ app_final.py                        l'agent complet, BUILD v9, port 3000
├─ app_tp.py                           l'agent pedagogue, BUILD v4, port 3002
├─ app_maths.py                        la demo maths, BUILD v3, port 3001
├─ banque_react.py                     22 snippets valides par le prof
├─ cours\                              la matiere (txt, docx, pdf, livres\, cache_web\)
├─ Tps\                                le TP (PDF, plan_tp.json, fiche_A.json, progres.csv)
├─ static\                             app.css + mathjax-tex-svg.js (hors-ligne)
├─ resultats\                          eval.csv, lien_public.txt, tunnel.log
├─ installer.bat, demarrer-agent.bat, partager.bat, build_exe.bat
└─ requirements.txt
```

B. Toutes les commandes, en une page

Quand	Commande	Attendu
Installation (une fois)	installer.bat	llama3.2 téléchargé, libs OK
Vérifier le moteur	ollama list	llama3.2:latest
Étape 1	python etape1.py	réponse, parfois fausse
Étape 2	python etape2.py	515 caractères, refus hors-matière
Étape 3	python etape3.py	[outil] lire_cours()
Étape 4	python etape4.py	page sur 127.0.0.1:3000
Étape 5	python etape5_web.py	quiz + score Python
Étape 6	python etape6_web.py	banque + correction
Étape 7	python etape6_web_doc.py	réponse citant un .docx
Étape 8	python etape8.py --demo	5 chemins, 1 seul appel LLM
Étape 9	python etape9_tokens.py	13, 11, 16 tokens
Étape 9 (éval)	python etape9c_eval.py	score dans resultats/eval.csv
Étape 10	python etape10_ressources.py	livre + sites
Agent complet	python app_final.py	BUILD v9, port 3000
TP	python app_tp.py	BUILD v4, port 3002
Maths (option)	python app_maths.py	BUILD v3, port 3001
Partager	partager.bat	URL trycloudflare avec ?k=
Partager le TP	python partager.py 3002	URL du TP
Lien ouvert	python partager.py --sans-cle	déconseillé en classe
Exe	build_exe.bat	dist\AgentReact\AgentReact.exe
Nettoyage	taskkill /F /IM python.exe	plus de serveur fantôme

C. Glossaire

Terme	Définition utile en atelier
LLM	Modèle de langage : un programme qui prédit la suite de mots. Il ne « sait » pas, il calcule une vraisemblance.
Token	Morceau de mot (environ 4 caractères). Un modèle compte en tokens, pas en caractères.

Terme	Définition utile en atelier
Fenêtre de contexte	Nombre maximal de tokens que le modèle peut lire en une fois (8 192 ici).
Hallucination	Réponse fluide et fausse, produite sans source. L'étape 1 la montre, l'étape 8 la bloque.
RAG	Recherche dans des documents, puis envoi des passages utiles au modèle. Le modèle ne lit pas le disque.
Garde	Filtre Python qui refuse une question avant l'appel au modèle.
Pastille	Étiquette de source affichée sous une réponse, extraite du texte réellement lu.
Tool-calling	Le modèle demande une fonction ; Python l'exécute et renvoie le résultat.
ReAct	Boucle raisonner → agir → observer, plafonnée ici à cinq tours.
AST	Arbre syntaxique d'un code : permet de repérer des noms inconnus et des fautes de frappe.
Bac à sable	Exécution d'un code étudiant dans un fichier temporaire, avec un délai maximal.
Tunnel	Connexion sortante qui rend un serveur local accessible par une URL publique, sans ouvrir de port.
Cache	Copie locale d'une page web, pour répondre sans wifi et prouver la source (cache).
OCR	Reconnaissance de texte sur une image : nécessaire pour les PDF scannés.

D. Checklist J-7

☐	Vérification	Critère
☐	Ollama installé et llama3.2 présent	ollama list
☐	Bibliothèques installées	python -c "import requests, pypdf, transformers; print('OK')"
☐	Le tokenizer gpt2 est en cache	python etape9_tokens.py affiche 13 tokens
☐	Les 10 étapes ont été lancées une fois	aucune erreur bloquante
☐	app_final.py répond et affiche une pastille	question Tailwind
☐	Le calendrier donne la bonne date	lundi 08h-12h, Mourad Sehboub
☐	La page est stylée sans wifi	CSS local, pas de CDN
☐	app_tp.py ouvre sur 3002	fiche A visible
☐	Le tunnel fonctionne	403 sans clé, 200 avec
☐	La clé USB contient dist + installer.bat	test sur un poste neuf
☐	Aucun python.exe fantôme	taskkill avant la séance
☐	Le document est imprimé ou ouvert	cette page suffit

E. Où est le code, et ce qui a changé

Ce document ne contient volontairement aucun code Python. La solution est dans les fichiers : etape1.py à etape10_ressources.py pour le parcours, app_final.py pour l'agent complet, app_tp.py pour le tuteur de TP, app_maths.py pour la démonstration de transfert.

> **CETTE VERSION DU 1ER NOVEMBRE 2026**
>
> etape8.py a été reconstruit : il n'existait pas dans le dossier. Il reprend la logique de app_final.py (lecture des documents, tri des passages, extraction des sources, calendrier, banque) et l'expose en console avec cinq démonstrations. Il a été testé avec Ollama allumé, puis en mode dégradé sans Ollama. requirements.txt verrouille désormais huggingface-hub<2, sans quoi l'étape 9 ne démarre pas.

F. La fiche à garder sous les yeux

LES CINQ QUESTIONS ET LES TROIS PHRASES

```
1. Que dit la matiere sur Tailwind ?          -> RAG + pastille .docx
2. Quand le cours 420-TT4-AS ?                -> calendrier, 0 LLM
3. Donne le code useReducer                   -> banque, 0 LLM
4. Explique useReducer ?                      -> garde, refus
5. Quelle est la recette de la tarte ?        -> garde, refus

Le modele ne lit pas vos documents : Python lit, trie, apporte.
Le modele propose, Python decide : notes, horaires, code viennent de fichiers.
Un agent qui repond a tout sans source est un moteur a vraisemblance, pas un agent de cours.
```